Promise

DANIELA GARCÍA QUINTERO

MARYHUG DURÁN PEDRAZA

ANGELO GAVIRIA MARÍN

ANDREA LIZCANO

EMMANUEL ARANGO VELÁSQUEZ

CLAN HAMILTON – Célula Nébula

PROMISE

"More human"

RIWI

MEDELLÍN

2026

Promise

Introducción

Este documento constituye la evidencia formal del proceso de Ingeniería de Requisitos aplicado durante el desarrollo de Promise, empresa emergente de automatización de procesos mediante agentes de inteligencia artificial, y de Sofía, su primer agente en producción.

Sofía es un sistema compuesto por cuatro microservicios que actúan como un agente unificado: el orquestador central (Sofía-Integral), el motor de llamadas de voz (Sofía Core), el agente de texto por WhatsApp (SofiaChat) y el dashboard de monitoreo (dashboardRiwicall). Aunque se desarrollaron como repositorios independientes, en producción los cuatro representan a un único agente con múltiples capacidades.

El proceso documentado aquí siguió las seis etapas canónicas de la Ingeniería de Requisitos: estudio de viabilidad, elicitación, análisis y negociación, especificación, validación y gestión. Cada etapa se presenta con su metodología, sus resultados y, donde aplica, con evidencia directa del sistema en producción.

1. Estudio de Viabilidad

Antes de iniciar el levantamiento formal de requisitos, el equipo realizó un estudio de viabilidad para determinar si el sistema propuesto era factible técnica y económicamente, y si generaba valor real para Riwi como cliente piloto.

1.1 Viabilidad Técnica

Se evaluaron los siguientes criterios:

Criterio	Análisis	Conclusión
Madurez de APIs de voz IA	ElevenLabs ofrece una API de síntesis y reconocimiento de voz con soporte para llamadas salientes vía Twilio, con documentación completa y SDKs disponibles.	Viable
Stack del equipo	El equipo domina Node.js (backend), Python/FastAPI (microservicios) y JavaScript vanilla (frontend). El stack elegido se alinea con las capacidades existentes.	Viable
Base de datos y persistencia	Supabase (PostgreSQL gestionado) provee la infraestructura de datos con SSL, conexiones pooled y API REST. No requiere infraestructura propia.	Viable
Integración de canales (WhatsApp)	Evolution API permite conectar una instancia de WhatsApp Business de forma programática. Alternativa de código abierto con amplia comunidad.	Viable

Criterio	Análisis	Conclusión
Complejidad de orquestación	Se evaluó N8N como orquestador visual. Tras pruebas técnicas, se descartó por limitaciones de latencia y control. Se optó por un orquestador propio en Node.js (ver sección 6).	Requirió ajuste
Tiempo de desarrollo disponible	El proyecto integrador tiene un ciclo de sprints definido dentro del entrenamiento de Riwi. La arquitectura de microservicios permite desarrollo paralelo por integrante.	Viable con priorización

1.2 Viabilidad Económica

Se realizó una comparación de costos entre el proceso manual de Riwi y el proceso automatizado con SofIA:

Dimensión	Proceso Manual (antes)	Con SofIA (después)
Capacidad diaria	75 llamadas por asistente de telemercadeo	1.200 llamadas por agente por jornada
Multiplicador de productividad	Línea base: 1x	16x respecto al proceso manual
Costo operativo estimado	Salario asistente + prestaciones + rotación + tiempo perdido en no-respuestas	Reducción del 86% en costo operativo frente al proceso manual
Escalabilidad	Lineal: más volumen = más personas contratadas	No lineal: el mismo agente escala con configuración, sin costo marginal por llamada adicional
Registro y trazabilidad	Manual, propenso a errores, sin visibilidad en tiempo real	Automático, transaccional, con dashboard en tiempo real
Riesgo de agotamiento del equipo	Alto: tareas repetitivas de bajo valor cognitivo consumían la jornada	Eliminado para el componente de marcación. El equipo humano se enfoca en excepciones.

Conclusión de viabilidad: el sistema es técnicamente factible con el stack y los recursos del equipo, y económicamente justificado con una reducción demostrada del 86% en costos operativos validada con datos reales de producción en Riwi.

2. Elicitación de Requisitos

La elicitación no comenzó con una sesión formal de levantamiento — comenzó con un problema observable. El equipo de Promise identificó directamente el cuello de

botella en el proceso de admisiones de Riwi y estructuró su recolección de información en tres técnicas complementarias.

2.1 Técnicas Utilizadas

Técnica	Descripción de la actividad	Resultado obtenido
Observación directa	El equipo observó el flujo de trabajo diario del equipo de admisiones de Riwi: cómo marcaban, qué registraban, cuánto tiempo tardaban por aspirante y cuántos no contestaban.	Identificación del cuello de botella: 27 intentos por aspirante, 75 llamadas/día por persona, registro manual propenso a errores.
Revisión de datos históricos	Se analizaron los datos de llamadas de cohortes anteriores: número de intentos promedio, tasa de contacto efectivo, tiempo total invertido por campaña.	Cuantificación del problema: la mayoría del esfuerzo se concentraba en aspirantes que nunca iban a contestar, con bajo retorno por llamada.
Sesiones de trabajo con coordinadores	Talleres con los coordinadores de admisiones para mapear el proceso actual (as-is), definir el proceso deseado (to-be) y establecer los escenarios de conversación que el agente debía manejar.	Definición de los 5 resultados posibles de una llamada: AGENDADO, PENDIENTE, NO_INTERESADO, NUMERO_INCORRECTO, BUZON_VOZ. Definición de franjas horarias y lógica de reintentos.
Prototipado conversacional	Se diseñaron guiones de conversación para los principales escenarios (confirmación, reprogramación, rechazo, número equivocado) antes de configurar el agente de voz en ElevenLabs.	System prompt inicial del agente y estructura del payload dinámico enviado a ElevenLabs con datos del candidato.

2.2 Partes Interesadas (Stakeholders)

Stakeholder	Rol en el proyecto	Necesidades principales identificadas
Coordinadora de admisiones	Usuario principal	Reducir tiempo en marcación manual. Visibilidad del estado de la campaña. Poder configurar parámetros sin depender de TI.
Equipo de admisiones Riwi	Usuario operativo	No tener que marcar manualmente. Recibir solo los casos que requieren atención humana. Registro automático de resultados.
Dirección Riwi	Sponsor / cliente	Escalabilidad del proceso de contacto sin aumentar headcount. Reducción de costos. Trazabilidad de resultados por cohorte.
Equipo Promise	Desarrolladores	Sistema modular, mantenible y desplegable por partes. API interna documentada. Configuración por cliente sin cambios de código.

Stakeholder	Rol en el proyecto	Necesidades principales identificadas
Aspirante (candidato)	Usuario final	Ser contactado en horarios convenientes. Poder confirmar o reprogramar de forma natural. No repetir información en cada llamada.

3. Análisis y Negociación de Requisitos

Con los requisitos recolectados, el equipo realizó un análisis de viabilidad técnica por requisito, identificó conflictos entre stakeholders y priorizó usando la metodología MoSCoW.

3.1 Tabla de Priorización MoSCoW

La siguiente tabla recoge todos los requisitos identificados con su clasificación MoSCoW, el microservicio responsable y el estado de implementación:

ID	Requisito	MoSCoW	Microservicio	Estado
RF-01	Motor de llamadas automatizadas salientes	Must Have	SofIA-Integral	Implementado
RF-02	Comprensión contextual del lenguaje natural en llamada	Must Have	SofIA Core (ElevenLabs)	Implementado
RF-03	Gestión automática de reintentos por franja horaria	Must Have	SofIA-Integral	Implementado
RF-04	Registro automático del resultado de cada llamada	Must Have	SofIA-Integral	Implementado
RF-05	Reserva atómica de cupos en eventos al agendar	Must Have	SofIA-Integral	Implementado
RF-06	Priorización de candidatos por score de afinidad (ci_total)	Must Have	SofIA-Integral	Implementado
RF-07	Dashboard básico de monitoreo de campañas	Must Have	dashboardRiwicall	Implementado
RF-08	Integración de los cuatro microservicios como sistema unificado	Must Have	Todos	Implementado
RF-09	Agente de chat multicanal (WhatsApp / Telegram / web)	Should Have	SofiaChat	Implementado
RF-10	Activación automática del chat tras 9/18/27 llamadas fallidas	Should Have	SofIA (bridge)	Implementado

ID	Requisito	MoSCoW	Microservicio	Estado
RF-11	Configuración de parámetros de campaña sin modificar código	Should Have	SofIA-Integral	Implementado (.env)
RF-12	Exportación de reportes de campaña	Should Have	dashboardRiwicall	Parcial
RF-13	Importación de candidatos desde Excel	Should Have	dashboardRiwicall	Implementado
RF-14	Análisis de sentimiento en llamadas	Could Have	N/A	No implementado v1
RF-15	Integraciones con CRMs externos	Could Have	N/A	No implementado v1
RF-16	Panel de analytics avanzado	Could Have	dashboardRiwicall	Básico implementado
RF-17	Tablero Kanban de seguimiento de candidatos	Could Have	dashboardRiwicall	Implementado
RF-18	Módulo de facturación o pagos	Won't Have v1	N/A	Fuera de alcance
RF-19	Soporte multilenguaje	Won't Have v1	N/A	Fuera de alcance
RF-20	Videollamadas	Won't Have v1	N/A	Fuera de alcance

3.2 Conflictos Identificados y Resolución

Conflicto	Stakeholders en tensión	Resolución adoptada
Activar chat muy pronto vs. demasiado tarde	Equipo de admisiones (quería activarlo rápido) vs. criterio técnico de no saturar WhatsApp	Se definió el umbral de 9 llamadas fallidas en el día como punto de activación. El número exacto (no ≥9 sino =9, 18, 27) evita activaciones múltiples el mismo día.
Usar orquestador visual (N8N) vs. orquestador propio	Velocidad de desarrollo (N8N más rápido) vs. control técnico (orquestador propio más flexible)	Se descartó N8N tras pruebas de latencia y control de colas. Se desarrolló orquestador propio en Node.js (documentado en SCRUM-25).
Granularidad de franjas horarias	Aspirantes preferían ser contactados en horarios específicos;	Se implementaron tres franjas fijas (mañana 7-12h, tarde 12-18h, noche 18-21h) con rotación automática tras cada llamada fallida.

Conflicto	Stakeholders en tensión	Resolución adoptada
	coordinadoras querían máxima cobertura	
Modularidad vs. simplicidad arquitectónica	Equipo técnico (prefería monolito por velocidad) vs. criterio de escalabilidad a otros clientes	Se optó por microservicios desde el inicio para validar el modelo de negocio de Promise (cada cliente puede necesitar solo algunos módulos).

4. Especificación de Requisitos

Los requisitos funcionales se especificaron mediante historias de usuario con criterios de aceptación verificables. Los requisitos no funcionales se documentaron como restricciones del sistema.

4.1 Requisitos Funcionales — Historias de Usuario

HU-01	Lanzamiento de campaña de contacto
Actor	Coordinadora de admisiones
Narrativa	*Como coordinadora de admisiones, quiero cargar una lista de aspirantes y lanzar una campaña de llamadas automatizadas, para que SofIA los contacte sin que mi equipo tenga que marcar manualmente cada número.*
Microservicio	SofIA-Integral / dashboardRiwicall
Criterios de aceptación	1. El sistema acepta un archivo con los datos de los aspirantes. 2. La campaña inicia automáticamente al configurar los parámetros. 3. El progreso es visible en tiempo real desde el dashboard. 4. Cada llamada queda registrada con su resultado.

HU-02	Gestión automática de reintentos
Actor	Coordinadora de admisiones
Narrativa	*Como coordinadora, quiero que el agente reintente a quienes no contestaron en horarios distintos y con priorización inteligente, para maximizar la tasa de contacto sin intervención manual.*
Microservicio	SofIA-Integral
Criterios de aceptación	5. El sistema programa reintentos en franjas horarias configurables. 6. Los aspirantes con más intentos fallidos tienen menor prioridad. 7. El número máximo de reintentos es configurable. 8. El sistema deja de reintentar al alcanzar el límite o al obtener respuesta.

Promise

HU-03	Conversación de voz con toma de decisiones
Actor	Aspirante contactado
Narrativa	*Como aspirante, quiero poder responder al agente de forma natural durante la llamada, para confirmar o reprogramar mi cita sin hablar con una persona ni usar una app.*
Microservicio	SofIA Core (ElevenLabs)
Criterios de aceptación	9. El agente comprende respuestas afirmativas, negativas y condicionadas en lenguaje coloquial. 10. Si el aspirante quiere reprogramar, ofrece opciones de horario disponibles. 11. El resultado se registra automáticamente en la base de datos. 12. La llamada cierra de forma coherente en cualquier escenario.

HU-04	Monitoreo de campañas en tiempo real
Actor	Administrador de campaña
Narrativa	*Como administrador, quiero ver en un tablero el estado actualizado de todas las llamadas, para tener visibilidad del proceso sin revisar registros manualmente.*
Microservicio	dashboardRiwicall
Criterios de aceptación	13. El dashboard muestra llamadas en curso, completadas, fallidas y pendientes. 14. Las métricas se actualizan en tiempo real. 15. Se puede filtrar por estado, fecha y campaña. 16. El reporte es exportable al finalizar.

HU-05	Atención por chat multicanal
Actor	Usuario final (aspirante)
Narrativa	*Como usuario, quiero comunicarme con SofIA por WhatsApp, Telegram o la web, para resolver dudas o completar procesos sin depender de una llamada.*
Microservicio	SofiaChat
Criterios de aceptación	17. El agente responde en menos de 3 segundos en cualquier canal. 18. Mantiene coherencia conversacional dentro de la sesión. 19. Escala a un agente humano cuando no puede resolver. 20. El historial queda registrado.

HU-06	Integración de microservicios
Actor	Desarrollador de Promise

Narrativa	*Como desarrollador, quiero que los cuatro módulos se comuniquen a través del orquestador central, para que operen como un agente unificado sin duplicar lógica entre servicios.*
Microservicio	SofIA-Integral (orquestador)
Criterios de aceptación	21. Cualquier evento en un microservicio actualiza el estado global en menos de 2 segundos. 22. Los servicios se despliegan y escalan de forma independiente. 23. La API interna está documentada. 24. El sistema maneja errores entre servicios sin pérdida de datos.

HU-07	Configuración de campaña por cliente
Actor	Administrador de Promise
Narrativa	*Como administrador, quiero configurar los parámetros de cada campaña (cliente, mensajes, horarios, reintentos) desde el sistema, para adaptar el agente a distintos clientes sin tocar el código fuente.*
Microservicio	SofIA-Integral (.env / config)
Criterios de aceptación	25. Los parámetros se configuran desde un panel o archivo de configuración. 26. Los cambios aplican sin reiniciar el sistema. 27. Cada cliente tiene perfil independiente.

HU-08	Rendimiento bajo volumen alto
Actor	CTO de Promise
Narrativa	*Como CTO, quiero que el sistema mantenga rendimiento estable procesando 2.400 llamadas por jornada, para garantizar el nivel de servicio comprometido con el cliente.*
Microservicio	SofIA-Integral
Criterios de aceptación	28. Los tiempos de respuesta no degradan más del 15% bajo carga máxima. 29. El sistema de colas gestiona la priorización sin pérdida ni duplicación de tareas. 30. Los logs permiten identificar cuellos de botella en tiempo real.

4.2 Requisitos No Funcionales

ID	Categoría	Descripción	Criterio de medición
RNF-01	Rendimiento	El sistema debe procesar hasta 2.400 llamadas por jornada sin	Degradación máxima del 15% bajo

ID	Categoría	Descripción	Criterio de medición
		degradación de tiempos de respuesta.	carga máxima simulada.
RNF-02	Disponibilidad	Operación continua durante campañas activas con tolerancia a fallos en servicios individuales.	El fallo de un microservicio no detiene el sistema completo.
RNF-03	Escalabilidad	La arquitectura de microservicios debe permitir escalar componentes de forma independiente.	Cada servicio puede redesplegarse sin afectar a los demás.
RNF-04	Seguridad	Los datos de aspirantes (nombre, teléfono, resultados) deben manejarse conforme a principios de protección de datos personales.	No se exponen datos en logs. Conexiones con SSL obligatorio.
RNF-05	Mantenibilidad	Cada microservicio documentado con README, endpoints y estructura de datos.	Un desarrollador nuevo puede entender y correr el servicio en menos de 30 minutos.
RNF-06	Idempotencia	El procesamiento de un webhook duplicado no debe generar registros duplicados en la base de datos.	Verificación de conversation_id antes de insertar en la tabla llamadas.
RNF-07	Concurrencia	El dispatcher debe manejar múltiples instancias sin colisiones en la cola de llamadas.	Uso de FOR UPDATE SKIP LOCKED y advisory locks en PostgreSQL.

Promise

5. Matriz de Trazabilidad de Requisitos

La matriz de trazabilidad vincula cada historia de usuario con el requisito funcional que la origina, el microservicio que la implementa, el archivo de código principal y el resultado de la validación en producción.

HU	RF	Descripción	Microservicio	Archivo principal	Validación
HU-01	RF-01	Llamadas automatizadas salientes	SofIA-Integral	src/schedulers/callDispatcher.js	Producción Riwi
HU-01	RF-06	Priorización por ci_total	SofIA-Integral	sql/fill_queue.sql	Producción Riwi
HU-02	RF-03	Reintentos por franja horaria	SofIA-Integral	src/postcall/rules.js	Producción Riwi
HU-03	RF-02	Comprensión lenguaje natural	SofIA Core	Panel ElevenLabs + payloadBuilder.js	Producción Riwi
HU-03	RF-05	Reserva atómica de cupos	SofIA-Integral	sql/postcall/reserve_event_slot.sql	Producción Riwi
HU-04	RF-07	Dashboard de monitoreo	dashboardRiwicall	src/js/loaders/dashboardView.js	Demo funcional
HU-05	RF-09	Agente de chat WhatsApp	SofiaChat	main.py / ai_agent.py	Demo funcional
HU-05	RF-10	Activación 9/18/27 fallidas	SofIA (bridge)	chatbot/chatbot.service.js	Producción Riwi
HU-06	RF-08	Integración microservicios	SofIA-Integral	src/index.js	Producción Riwi
HU-07	RF-11	Config por cliente sin código	SofIA-Integral	src/config/env.js	Producción Riwi
HU-08	RNF-01	Rendimiento 2.400 llamadas	SofIA-Integral	callDispatcher.js (concurrencia)	Medido en producción
HU-08	RNF-07	Concurrencia sin colisiones	SofIA-Integral	sql/dispatcher/select_pending_for_lock.sql	FOR UPDATE SKIP LOCKED
HU-01	RF-04	Registro del resultado	SofIA-Integral	src/postcall/service.js	Producción Riwi
HU-05	RNF-06	Idempotencia de webhooks	SofIA-Integral	sql/postcall/find_call_by_conversation_id.sql	Revisión de código ☑

HU	RF	Descripción	Microservicio	Archivo principal	Validación
HU-02	RNF-04	Seguridad datos aspirantes	Todos	src/db/pool.js (SSL)	SSL obligatorio ☑

6. Validación de Requisitos

La validación de requisitos en Promise adoptó un enfoque híbrido: revisión técnica interna durante el desarrollo y validación empírica con datos reales de producción. Esta última es la evidencia más contundente de que los requisitos capturaron correctamente el problema del cliente.

6.1 Validación en Producción — Datos Reales de Riwi

SofIA fue desplegado para campañas reales del proceso de admisiones de Riwi. Los datos obtenidos en producción validan directamente los requisitos más críticos del sistema:

Requisito validado	Métrica esperada	Resultado en producción	Estado
RF-01 — Capacidad de llamadas	> 75 llamadas diarias (línea base manual)	1.200 llamadas diarias por agente por jornada (16x el proceso manual)	Superado
RNF-01 — Costo operativo	Reducción significativa vs. proceso manual	86% de reducción en costo operativo frente al proceso manual medido en Riwi	Validado
RF-04 — Registro de resultados	100% de llamadas con resultado registrado	Todos los resultados registrados automáticamente en tabla llamadas vía webhook postcall	Validado
RF-05 — Reserva de cupos	Sin doble-agendamiento de un mismo cupo	Reserva atómica con UPDATE ... WHERE estado = DISPONIBLE AND inscritos < capacidad previene sobreventas	Validado
RNF-06 — Idempotencia	Webhooks duplicados no generan registros duplicados	Verificación de conversation_id en tabla llamadas antes de procesar cada webhook entrante	Validado
RNF-07 — Concurrencia	Múltiples instancias sin colisión en cola	FOR UPDATE SKIP LOCKED garantiza que dos instancias no	Validado

Requisito validado	Métrica esperada	Resultado en producción	Estado
		procesen el mismo registro simultáneamente	

6.2 Revisión Técnica Interna

Durante el desarrollo se realizaron las siguientes actividades de validación interna:

- Pruebas de webhook con payloads reales: se verificó que cada resultado (AGENDADO, BUZON_VOZ, NO_INTERESADO, NUMERO_INCORRECTO, PENDIENTE) producía exactamente los cambios esperados en la base de datos.
- Scripts de diagnóstico: Se incluye 14 scripts npm de utilidades (npm run bot:check, db:events, logs:view) que permitieron verificar el estado del sistema en cualquier momento del desarrollo.
- Revisión de idempotencia: se simuló el envío doble del mismo webhook y se verificó que solo se creaba un registro en la tabla llamadas.
- Pruebas de carga conceptual: se simuló el llenado de la cola con 1.000 candidatos y se verificó que el advisory lock impedía inserciones duplicadas en cola_llamadas.
- Validación de conversaciones: se probaron manualmente los 4 escenarios de conversación del agente de chat (agendado, pendiente, no interesado, número equivocado) verificando que el estado resultante en la BD era correcto en cada caso.

7. Gestión de Requisitos y Control de Cambios

Durante el ciclo de desarrollo, varios requisitos y decisiones técnicas evolucionaron respecto al diseño inicial. A continuación, se documentan los cambios más significativos con su justificación y el impacto en el sistema.

7.1 Registro de Cambios

ID	Tipo	Cambio realizado	Justificación	Impacto
SCRUM-25	Decisión arquitectónica	Descarte de N8N como orquestador. Reemplazo por orquestador propio en Node.js.	N8N presentó limitaciones de latencia en la comunicación entre servicios y falta de control granular sobre la lógica de priorización de colas.	Alto. Requirió desarrollar callDispatcher.js, fillQueueRunner.js y todo el módulo de schedulers desde cero. Resultado: mayor control, menor latencia.

ID	Tipo	Cambio realizado	Justificación	Impacto
RF-03 v2	Evolución de requisito	La lógica de reintentos evolucionó de "número fijo de reintentos" a "rotación por franjas horarias" con priorización dinámica.	Las pruebas con datos reales mostraron que contactar al mismo candidato repetidamente en la misma franja horaria tenía tasa de éxito muy baja.	Medio. Se agregaron las columnas franja_actual e intentos_franja_actual a la tabla candidatos y se modificó update_candidate_retry.sql.
RF-10 nuevo	Requisito emergente	Se identificó la necesidad de un canal de fallback por WhatsApp para candidatos que nunca contestaban el teléfono.	Análisis de datos históricos reveló que un segmento significativo de aspirantes prefería comunicación por texto o no contestaba llamadas.	Alto. Implicó crear el repositorio SofiaChat completo y la lógica de activación shouldTriggerChatbot() con el umbral 9/18/27.
RNF-06 nuevo	Requisito no funcional emergente	Se agregó el requisito de idempotencia de webhooks (no procesamiento doble del mismo conversation_id).	En pruebas con ElevenLabs se observó que en condiciones de red inestable el mismo webhook podía llegar dos veces.	Bajo. Se agregó la verificación FIND_CALL_BY_CONVERSATION_ID al inicio de processPostCallResult() sin cambios arquitectónicos.
RF-02 scope	Ajuste de alcance	El modo FAQ de SofiaChat (respuesta a mensajes orgánicos de WhatsApp) se agregó al alcance durante el sprint final.	Se identificó que usuarios podían escribir directamente al número de WhatsApp sin haber sido contactados previamente por voz.	Bajo. Se agregó generar_prompt_faq() y la bandera es_faq en sesiones_activas. No afecta la base de datos.

ID	Tipo	Cambio realizado	Justificación	Impacto
RF-13 nuevo	Requisito emergente	Importación de candidatos desde Excel directamente en el dashboard sin necesidad de backend.	Las coordinadoras de admisiones trabajan con Excel como herramienta principal y no tenían acceso directo a la base de datos.	Medio. Se integró XLSX.js en dashboardRiwicall y se desarrolló la normalización de columnas del Excel.

7.2 Trazabilidad del Backlog SCRUM

El equipo gestionó el desarrollo bajo metodología SCRUM usando Jira (espacio Nébula). El sprint final concentra los entregables de cierre del proyecto integrador:

ID	Tarea	Estado	Vincula
SCRUM-17	Mockup Figma del sistema	En curso	RF-07
SCRUM-18	README de repositorios	Por hacer	RNF-05
SCRUM-19	Integración de microservicios	Por hacer	RF-08
SCRUM-20	Documento técnico mínimo (PDF)	En curso	Todos
SCRUM-21	Documentación técnica complementaria	Por hacer	RNF-05
SCRUM-22	Video o demo de presentación	Por hacer	—
SCRUM-23	Ingeniería de requisitos (este documento)	En curso	Todos
SCRUM-24	Landing page Promise + redirecciones a RiwiCall	Por hacer	RF-07
SCRUM-25	Decisión arquitectónica: descartar N8N	Completado	RF-08
SCRUM-26	Code Book del proyecto	Completado	RNF-05

La concentración de entregables de documentación en el sprint final es coherente con la estrategia del equipo: priorizar la construcción del producto en sprints anteriores y consolidar la documentación al cierre del ciclo. Esta decisión es en sí

misma una evidencia del proceso iterativo de SCRUM — se entregaron incrementos funcionales antes de la documentación formal.

8. Conclusiones

El proceso de Ingeniería de Requisitos aplicado en Promise demuestra que es posible combinar rigor metodológico con la agilidad propia de un startup en formación. Las principales conclusiones del proceso son:

- La elicitación basada en observación directa y datos históricos fue más efectiva que las sesiones formales de levantamiento. El problema de Riwi era observable sin necesidad de entrevistas estructuradas.
- La priorización MoSCoW permitió al equipo entregar valor real en producción (los "Must Have") sin bloquearse en funcionalidades deseables, pero no críticas.
- Dos requisitos cruciales emergieron durante el desarrollo y no estaban en el diseño inicial: la activación del chat de WhatsApp tras 9/18/27 llamadas fallidas (RF-10) y el requisito de idempotencia de webhooks (RNF-06). Esto confirma que la gestión continua de requisitos es tan importante como la elicitación inicial.
- La decisión de descartar N8N como orquestador (SCRUM-25) es el ejemplo más claro de cómo el proceso iterativo de SCRUM permitió al equipo ajustar decisiones arquitectónicas basándose en evidencia real, no en supuestos.
- La validación en producción con datos reales de Riwi (16x productividad, 86% reducción de costos) es la evidencia más sólida de que los requisitos capturaron correctamente el problema del cliente.
- La arquitectura de cuatro microservicios no es solo una decisión técnica: es una decisión de negocio que permite a Promise ofrecer a futuros clientes exactamente los módulos que necesitan, sin obligarlos a adoptar el sistema completo.